

```

/>
</form></body>
</html>

```

In the above code, the enctype feature of the <form> tag focuses on the content category to be used for submitting the form. To insert a binary data in an HTML form, say, the content of a file, the “multipart/form-data” should be used. The type= “file” in the above code implies that the input should be processed as a file.

See the upload script below:

The file name upload_file.php includes the code for uploading a file:

```

Example:
<?php
if ($_FILES["file"]["error"] > 0)
{
    echo "Error: " . $_FILES["uploadedfile"]["error"]
    . "<br />";
}
else
{
    echo "Uploaded File: " . $_FILES["uploaded-
file"]["name"] . "<br />";
    echo "Uploaded File Type: " . $_FILES["uploaded-
file"]["type"] . "<br />";
    echo "Uploaded File Size: " . ($_FILES["uploaded-
file"]["size"] / 1024) . " Kb<br />";
    echo "Uploaded File Stored in: " .
$_FILES["uploadedfile"]["tmp_name"];
}
?>

```

With the use of global PHP \$_FILES array, a file can be successfully uploaded from a client computer to the remote server. The form’s first parameter often remains input name and the second index varies from name, type, size, tmp_name or error. Look at the following code:

```

$_FILES["uploadedfile"]["name"] - uploaded file
name

```